MONGODB QUERY OPTIMIZATION

In this document, the query optimization process of mongodb query is described with different methods and execution results. Please check the below query.

```
db.Person.find({
  $or: [
    { 'properties.name: /usmanova/i },
    { 'properties.alias': /usmanova/i },
    { 'properties.description': /usmanova/i },
    { 'properties.keywords': /usmanova/i },
  ]
})
```

We can identify below properties of this query.

- 4 different filters with specific search criteria.
- $or is used.
- Data is searched and fetched from text fields.
- Case in-sensitive text search is used.
- Searched word/phrase can be from any position of the field value.
- Simple MongoDB 'find' is used.

After the execution of query without index we have identified that average run-time if query is around 4.5 seconds, which is quite high. So, we have created compound index using the query below on given 4 fields. Please check index query below.

```
Db.Person.createIndex([{
                        'properties.name:1,
                        'properties.alias':1,
                        'properties.description':1,
                        'properties.keywords':1
  })
```

We have found that after using the index in query execution, run time is increased to an average of 7.3 seconds approximately. 2x Increment of scanned key is the reason behind this incident. So basic index is not useful here. We have tried single fields index too but it is still not useful in this case. Using of 'i' is the bottle neck as it is preventing the index to create range for results.

In this case, Text index is savior here.Text indexes support text search queries on fields containing string content. Text indexes reduce run time when searching for specific words or phrases in text fields. A collection can have **one** text index only, but that index can cover multiple fields. Text indexes support $text query operations on deployments. To perform text searches, we must create a text index and use the $text query operator. Text indexes store individual words of a text string. They do not store phrases or information about the proximity of words in the documents. As a result, queries that specify multiple words run faster when the entire collection fits in RAM.

Below query will create a desired text index on collection Person.

```
db.Person.createIndex(
{
                "properties.name": 'text',
                "properties.alias" : 'text',
                "properties.description" : 'text',
                "properties.keywords" : 'text',
}
                    )
```

By using the above query, we can create an index on 4 sub fields of 'properties'. After creating the above index, we have observed that execution time of the query is reduced to average 0.10 Seconds approximately. Examined keys and documents are drastically reduced to 16 and 32 only which resulted very less run time.

Here, we cannot use same query if we want to use text index for query performance. Hence, the query design is changed and use of text search is added in query Please check below query.

```
db.Person.find({
$and:[{$text: {$search: "usmanova"}},
{$or: [
{ 'properties.name': /usmanova/i },
{ 'properties.alias': /usmanova/i },
{ 'properties.description': /usmanova/i },
{ 'properties.keywords': /usmanova/i },
]}
]})
```

Please check execution stats of above query with use of given text index.

```
"executionStats" : {
"executionSuccess" : true,
"nReturned" : 16.0,
"executionTimeMillis" : 1.0,
"totalKeysExamined" : 16.0,
"totalDocsExamined" : 32.0
```

Thank You!